

HW 2-B

Course: CSCE 703

UIN: 937006978

Email: srijapentyala@tamu.edu

GitHub: <https://github.com/srijapentyala/hw2b-juice-shop-login>

Part 1: Secure Feature Design

I ran Juice Shop locally. The table records three bugs I reproduced and the registration control that stops each one.

Vulnerability	Exploit Scenario	Security Mitigation
SQL Injection	I typed ' OR 1=1-- and a junk password. Juice Shop signed me in as admin@juice-sh.op and awarded Login Admin. The email is concatenated into the query, so the injected clause is treated as SQL.	A registration insert must use a parameterized query. The email stays a bound value, so it cannot add an administrator row or change the statement.
Cross-Site Scripting	Search writes the query into the page as HTML. My term rendered as a yellow XSS heading instead of text. A tag in a registered username would run the same way.	Encode the username on output and assign it with <code>textContent</code> . Send <code>Content-Security-Policy: script-src 'self'</code> so a tag is shown and does not run.
Weak Password Storage	The administrator password is admin123, and the login token includes its MD5 hash. Password Strength solved only when I used that password, not injection.	Reject common passwords. Store a bcrypt hash with a unique salt and cost 12, and never return the hash.

How These Measures Prevent Attacks

A parameterized query sends the email as data, so ' OR 1=1-- cannot change the statement.

Encoding the username, and refusing inline script, means a tag in the name is displayed as text.

bcrypt at cost 12 is slow and uses a new salt for every password, so the MD5 in the Juice Shop token cannot be reused.

```
bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
bcrypt.checkpw(password.encode(), stored_hash)
```

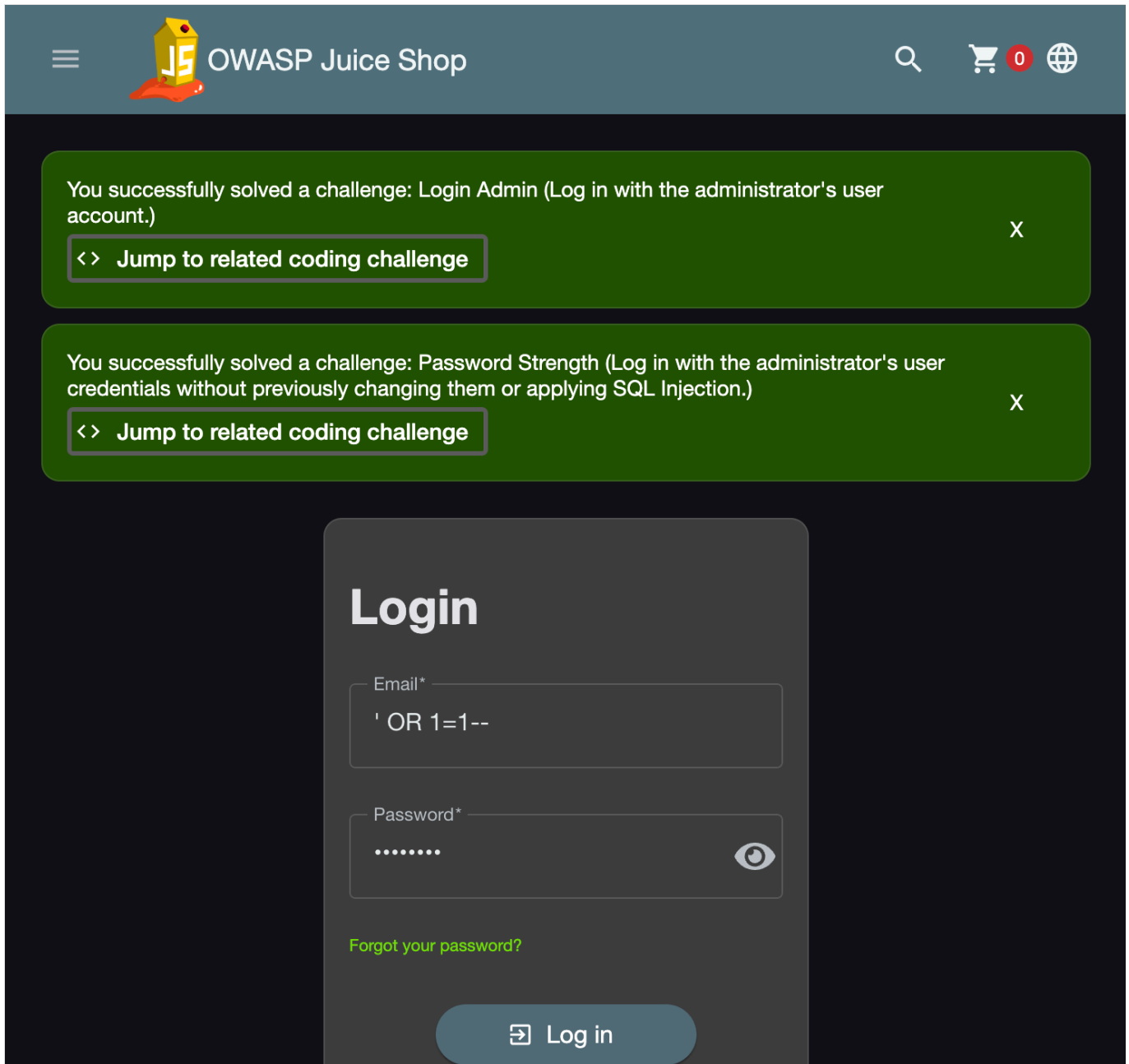


Figure 1. The injected email is still in the form. Juice Shop awarded Login Admin for that login, and Password Strength for admin123.

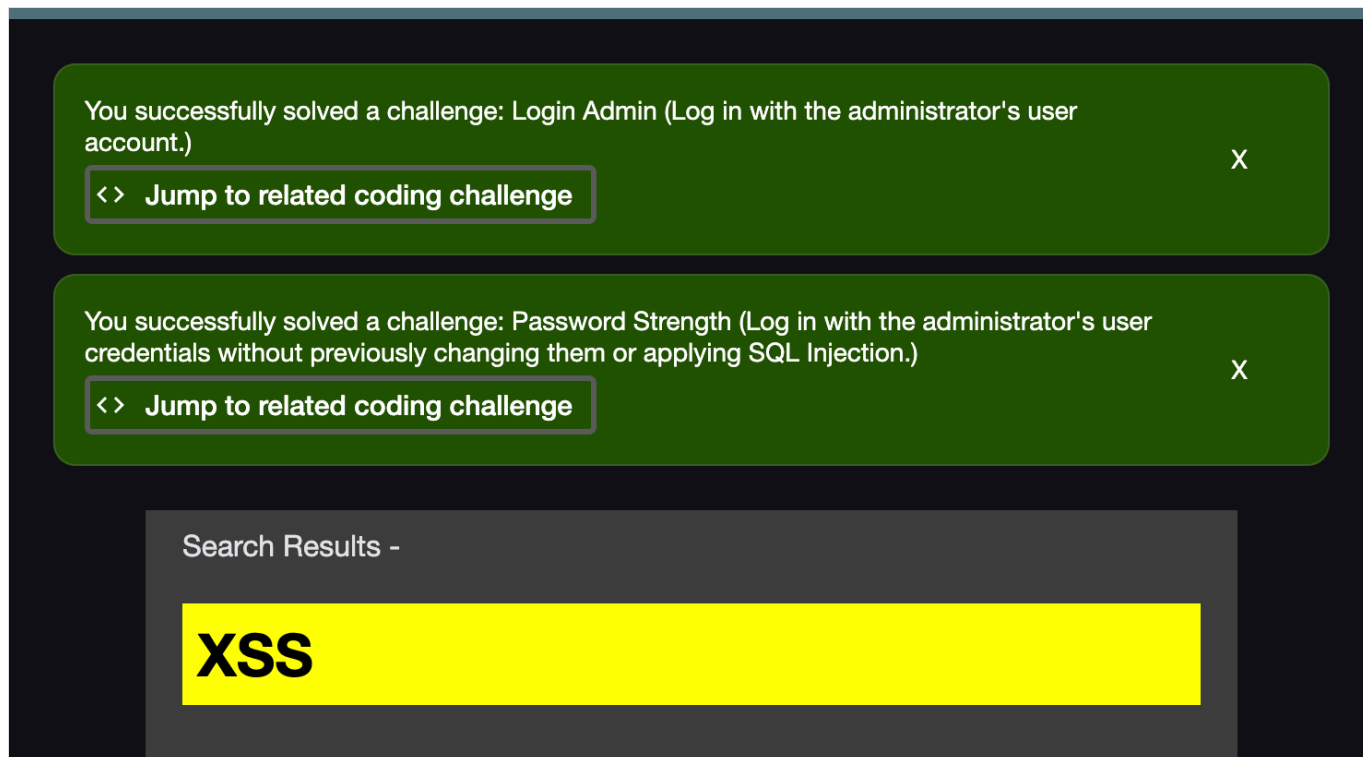


Figure 2. The search term was parsed as HTML. The tags never appear as text.

Part 2: Simple Front-End Form

GitHub: <https://github.com/srijapentyala/hw2b-juice-shop-login>

I built an HTML and JavaScript login form with Juice Shop's two fields, email and password. `login.js` blocks an empty submit, requires "@" in the email, and requires at least 8 characters. Those errors stay in the browser. A form that passes is sent as JSON to `POST /login`. `server.py` repeats the same three checks, then compares the password with a PBKDF2-HMAC-SHA256 hash in constant time. A wrong password and an unknown email get the same message. Nothing is placed in a SQL string. Run `python3 server.py` and open `http://127.0.0.1:8080`. The demo account is `demo@juice-sh.op` / `JuiceShop1`.

The image shows the OWASP Juice Shop login form. It has a dark header with the logo and name. The form itself is white and contains two input fields: "Email" and "Password". Below the fields is a green "Log in" button. The form is centered on a light grey background.

Figure 3. Login form before submission.

The image shows the same OWASP Juice Shop login form as in Figure 3, but now it displays an error message. Below the "Log in" button, the text "Email and password are required." is shown in red. The rest of the form and header are identical to the previous screenshot.

Figure 4. The browser rejects an empty submission.

Login

Email

Password

Log in

Logged in.

Submitted email: demo@juice-sh.op

Figure 5. The demo account passes both the browser check and the server check.

Part 3: Exploiting a Vulnerability

I repeated the Juice Shop attacks on `http://127.0.0.1:8080` after `python3 server.py`.

For SQL injection I entered ' OR 1=1--@juice-sh.op and password1, then clicked Log in. The format checks passed. The server answered "Invalid email or password." There is no SQL string to rewrite, so the login failed.

Login

Email

Password

Log in

Invalid email or password.

Submitted email: ' OR 1=1--@juice-sh.op

Figure 6. The SQL fragment is stored as an email address. Login fails.

XSS succeeds on this repository. `login.js` writes the submitted email with `innerHTML`. I entered `<img src=x onerror="document.getElementById('message').textContent='JavaScript executed'">@a.co` and `password1`, then clicked Log in. The image failed, the handler ran, and the status line became “JavaScript executed.” The login itself still failed.

Login

Email

Password

Log in

JavaScript executed

Submitted email: @a.co

Figure 7. The status line was written by the image handler, not by the login code.

Fix: assign that line with `textContent`, and send `Content-Security-Policy: script-src 'self'`, so the email is text and an inline handler cannot run.